

Lab 8: Deployment View & Quality Attribute Analysis (ATAM)

This final lab shifts the focus from coding to architecture documentation and quality attribute evaluation. Students will design the physical deployment diagram of the Microservices Architecture and use a simplified Architecture Trade-off Analysis Method (ATAM) to evaluate how the chosen architecture meets the Non-Functional Requirements (NFRs).

Objectives

1. Create a UML Deployment Diagram to visualize the physical installation environment of the Microservices system (Deployment View).
2. Conduct a simplified ATAM method, focusing on Scalability and Availability.
3. Compare the Monolithic Architecture and the Microservices Architecture based on the selected Quality Attributes within the context of the HRM system.

Tool & Technology Setup

This lab is primarily focused on documentation and diagramming.

Tool	Purpose	Installation/Setup Guide
draw.io (Diagrams.net)	Create UML Deployment Diagrams and visualize concepts.	Access online via a web browser.
Google Docs/Word	Document the ATAM analysis results and comparison matrix.	Standard word processing software.
HRM Architecture (from docker-compose file)	Conceptual mental model for component deployment.	No coding required, but a solid understanding of the project's docker-compose.yml file is needed.

Activity Practice 1: UML Deployment Diagram

Goal: Model the physical allocation of software components (artifacts) to execution environments (nodes - hardware/containers).

Step-by-Step Instructions (Using draw.io):

1. **Identify Nodes (Physical/Logical Environments):** In UML, a Node represents a computational resource (server, container, device). We will define them based on the project's actual Docker Compose configuration.
 - Action: Create the main nodes using the "Node" shape (a 3D box):
 - **Client Device:** Represents the Employee/Manager's web browser.
 - **Gateway Node (Internal Network):** The environment containing the routers.
 - **Application Cluster Node:** The execution environment for the Node.js Backend services.
 - **Database Cluster Node (Polyglot Persistence):** The specialized high availability (HA) data storage environment.
2. **Place Artifacts (Software Components):** An Artifact represents a specific piece of software (e.g., Docker Image). Place the following artifacts into their respective Nodes:
 - In Client Device: **React.js Frontend Artifact** (SPA).
 - In Gateway Node: **Nginx Load Balancer Artifact** and **Kong API Gateway Artifact**.
 - In Application Cluster Node: **Auth Service Artifact**, **Leave Service Artifact**, **Salary Service Artifact**, **Noti Service Artifact**.
 - In Database Cluster Node:
 - **PostgreSQL HA** (Spilo + Patroni + HAProxy)
 - **MongoDB Replica Set** (rs0)
 - **Redis Cache/Broker**
3. **Draw Associations (Network Communication):** Use dashed arrows to represent network communication.
 - Action: Draw the following associations:
 - Client Device $\rightarrow$ Nginx: HTTP/HTTPS (Web Communication).
 - Nginx $\rightarrow$ Kong API Gateway: HTTP (Traffic Forwarding).
 - Kong $\rightarrow$ Auth / Leave / Salary Service: Internal HTTP/REST (API Routing).

- All Backend Services $\rightarrow$ PostgreSQL HA & MongoDB: TCP/IP (Data Read/Write).
- Leave/Auth Service $\rightarrow$ Redis: Pub/Sub Protocol (Event Publishing).
- Redis $\rightarrow$ Noti Service: Pub/Sub Protocol (Event Subscribing for Email/Socket).

Activity Practice 2: Quality Attribute Analysis (Simplified ATAM)

Goal: Evaluate architectural decisions based on key non-functional requirements (Scalability and Availability) using a structured method.

Step-by-Step Instructions:

1. **Define Scenarios (Test Cases):** For each Quality Attribute, define a scenario that poses a challenge to the HRM system.
 - **Scalability Scenario (SS1):** "At 8:00 AM on the 1st day of the month, all 5,000 company employees log in simultaneously to check in and view their Net Salary. The system must handle a 20x load spike without crashing."
 - **Availability / Fault Tolerance Scenario (AS1):** "The Notification Service (NodeMailer) hangs due to Email provider API limits. The system must still allow employees to submit Leave Requests and allow Managers to approve them normally."
2. **Evaluate Architectures against Scenarios:** Analyze how the Monolithic Architecture compares to the Microservices Architecture (that your team built) in handling these scenarios.

Quality Attribute	Scenario	Monolithic Approach	Microservices Approach (Your Project)
Scalability	SS1 (Sudden spike in Salary/Attendance access)	Response: Must replicate the entire application (including unused Leave, Notification parts). Causes wasteful	Response: Only need to scale (increase container count) specifically for salary_service and auth_service. The Database uses PostgreSQL HA (Spilo) which automatically routes read queries to Replica

		RAM/CPU consumption. Filtering massive payroll data might bottleneck the entire flow.	nodes, ensuring extremely fast and cost-effective payroll calculation/read performance.
Availability	AS1 (Notification Service crashes)	Response: If email sending logic is tightly coupled in a monolithic block, when the email function hangs (timeout), the POST /api/leaves process also hangs. The employee clicks submit, but the web page freezes, and data is not saved to the DB.	Response: Thanks to the Event-Driven Architecture (EDA - Lab 7), leave_service only saves data to the DB, "shoots" a message to Redis Pub/Sub, and immediately returns a 200 OK code. The noti_service crashing does not affect the request creation flow. Excellent Fault Isolation.

3. **Identify Trade-offs:** Based on the evaluation, explicitly state the architectural trade-offs.

- **Trade-off:** The Microservices Architecture provides superior Scalability and Fault Tolerance (Isolation). However, it introduces immense **Complexity** in deployment and operations. Specifically, instead of running 1 app and 1 DB, Group 05 has to operate and maintain Nginx, Kong, a HAProxy+Patroni cluster for Postgres, a MongoDB Replica Set, and Redis (totaling over 15 containers in [docker-compose](#)). The error debugging process (Traceability) across services is also much more difficult compared to a Monolith.

Documentation & Submission

Prepare the final documentation artifacts for this lab:

1. **UML Deployment Diagram:** The diagram created in Practice 1, clearly showing the Nodes (Nginx, Services, Postgres HA, Mongo, Redis), Artifacts, and communication links.
2. **ATAM Analysis Table:** The completed table from Practice 2 comparing the two architectures against the HR business scenarios (SS1 and AS1).
3. **Trade-off Statement:** A brief paragraph summarizing the identified architectural trade-offs (Advantages vs. Infrastructure Complexity)